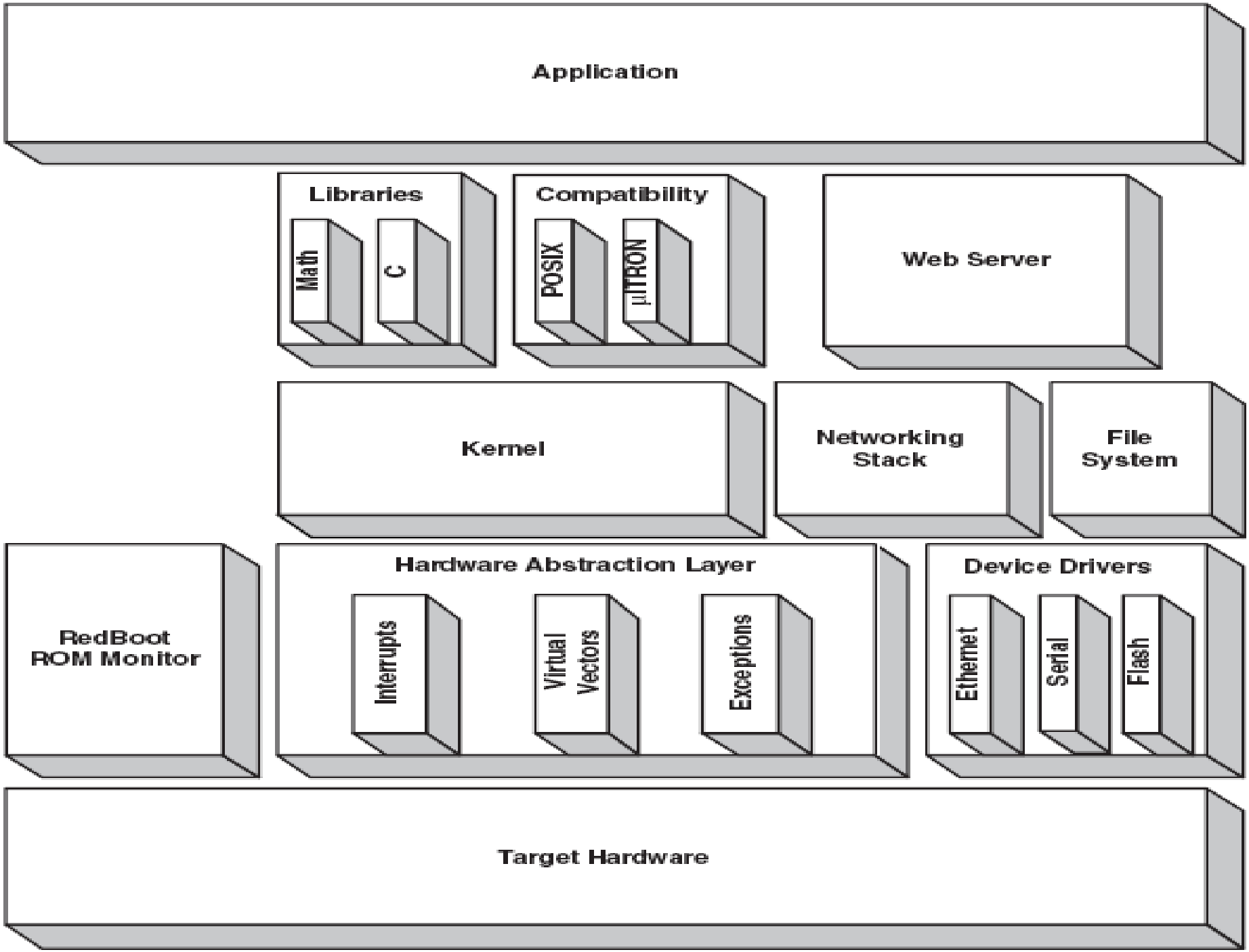
The eCos real-time operating system

an open source tool to create embedded kernels and applications


Application
Libraries
Math
C
Compatibility
POSIX
µITRON
Web Server
Kernel
Networking Stack
File System
RedBoot ROM Monitor
Hardware Abstraction Layer
Interrupts
Virtual Vectors
Exceptions
Device Drivers
Ethernet
Serial
Flash
Target Hardware

Configuration System

- It is the ‘heart’ of eCos.
- Select only the packages that are necessary through configuration.
- This reduces the footprint of the application.

- eCos uses compile-time control methods.
- This allows the application writer control over individual lines of code in the packages.
- The control methods are implemented through C Preprocessor

Example of Configuration

```
#if  defined(CYGDBG_HAL_DEBUG_GDB_INCLUDE_STUBS)
     && !defined(CYGPKG_CYGMON)

  if (__mem_fault_handler) {
          regs->pc = (CYG_ADDRWORD)__mem_fault_handler;
          return;
   }

    _hal_registers = regs;
    __handle_exception();
```

Example of Configuration

```
#elif defined(CYGFUN_HAL_COMMON_KERNEL_SUPPORT)
        && defined(CYGPKG_HAL_EXCEPTIONS)

cyg_hal_deliver_exception( regs->vector>>8,
    (CYG_ADDRWORD)regs );

#else
        CYG_FAIL("Exception!!!");
```

eCos Components

- The following are the core components :
  - Hardware Abstraction Layer
  - Real-time kernel.
  - ISO C and math libraries
  - Device drivers
  - GNU Debugger (GDB) support

- The real-time kernel is the central core component.

eCos API

- eCos supports the following standard API
  - µitron
  - POSIX
  - Embedded Linux API compatible with EL/IX.
  - It’s own native API.

Hardware Abstraction Layer (HAL)

- The HAL is a software layer.
- It provides a platform independent API for platform specific functionality.
- Enhances portability of code.

Example Implementation for ARM architecture

```
#define HAL_ENABLE_INTERRUPTS()  \
        asm volatile {                  \
                "mrs r3,cpsr;"          \
                "bic r3,r3,#0xc0;" \
                :                       \
                :                       \
                : "r3"                  \
        };
```

Example implementation for PowerPC Architecture

```
#define HAL_ENABLE_INTERRUPTS()                 \

        CYG_MACRO_START                         \
                cyg_uint32 tmp1, tmp2;          \
                asm volatile (                  \
                "mfmsr %0;"                     \
                "ori %1,%1,0x8000;"             \
                "rlwimi %0,%1,0,16,16;"         \
                "mtmsr %0;"                     \
                : "=r" (tmp1), "=r" (tmp2));    \
        CYG_MACRO_END
```

Example Implementation

- For both the platforms the underlying implementation of the macro HAL_ENABLE_INTERRUPTS() is different.

- But the API is the same macro

  HAL_ENABLE_INTERRUPTS()

Example Scenario

- Generally on being interrupted, all interrupts are disabled.

- Bad idea :
  - Enable interrupts at the end of ISR.
  - Disadv. : System loses predictability.

- Good idea :
  - Enable all interrupts at the start of ISR.
  - Adv. : interrupts can be pre-empted

The Kernel

- The Kernel is the core to the eCos system.

- Provides standard functionality like
  - interrupt and exception handling
  - scheduling
  - threads
  - synchronization

Kernel API

- The kernel provides a C API for direct interfacing to the kernel.
- The kernel API does not return error codes as is usual.
- Instead it provides a number of assertions that can be enabled or disabled.

Assertions available

- CYG_FAIL (*diag_message*)
  Does not accept a condition as its first argument.

- CYG_ASSERT (*condition, diag_message*)
  Accepts a condition as it's first argument.

- CYG_ASSERTC (*condition*)
  Compact version of the above assertion

Assertions

- The first two assertions output a diagnostic message that is given as parameter.
- CYG_FAIL outputs the messages irrespective of any conditions.
- CYG_ASSERTC() macro does not output any diagnostic messages.

Exception Handling

- Exception handling can be done in two ways :
  - HAL + Kernel Exception Handling
  - Application Exception Handling

- HAL + Kernel Exception Handling is the default option.

HAL + Kernel Exception Handling

- Uses a Vector Service Routine (VSR).
- It is an array of pointers to exception handler routines.

- HAL does basic interrupt processing like saving the context etc …

- Then control goes to kernel for further processing if required.

Application Exception Handling

- Applications can provide their own VSR when an exception occurs.
- VSR's ***must*** be written in assembly language.
- HAL_VSR_GET and HAL_VSR_SET are macros provided to give access to VSR table.

Interrupt Processing

- Provides ISR's and DSR's
- ISR's perform most common tasks. They are small and execute quickly.
- DSR's perform additional processing if necessary.

Interrupt Processing

- In ISR's calling synchronization primitives is not allowed.
- They are allowed in DSR.
- DSR must not make any synchronization calls that block.

Interrupt Processing

- Synchronization primitives are not allowed inside the ISR's because
  - ISR's must be fast and bounded by time.
  - If due to some reason a synchronization primitive causes the task to wait or sleep then it is not acceptable.

Scheduler

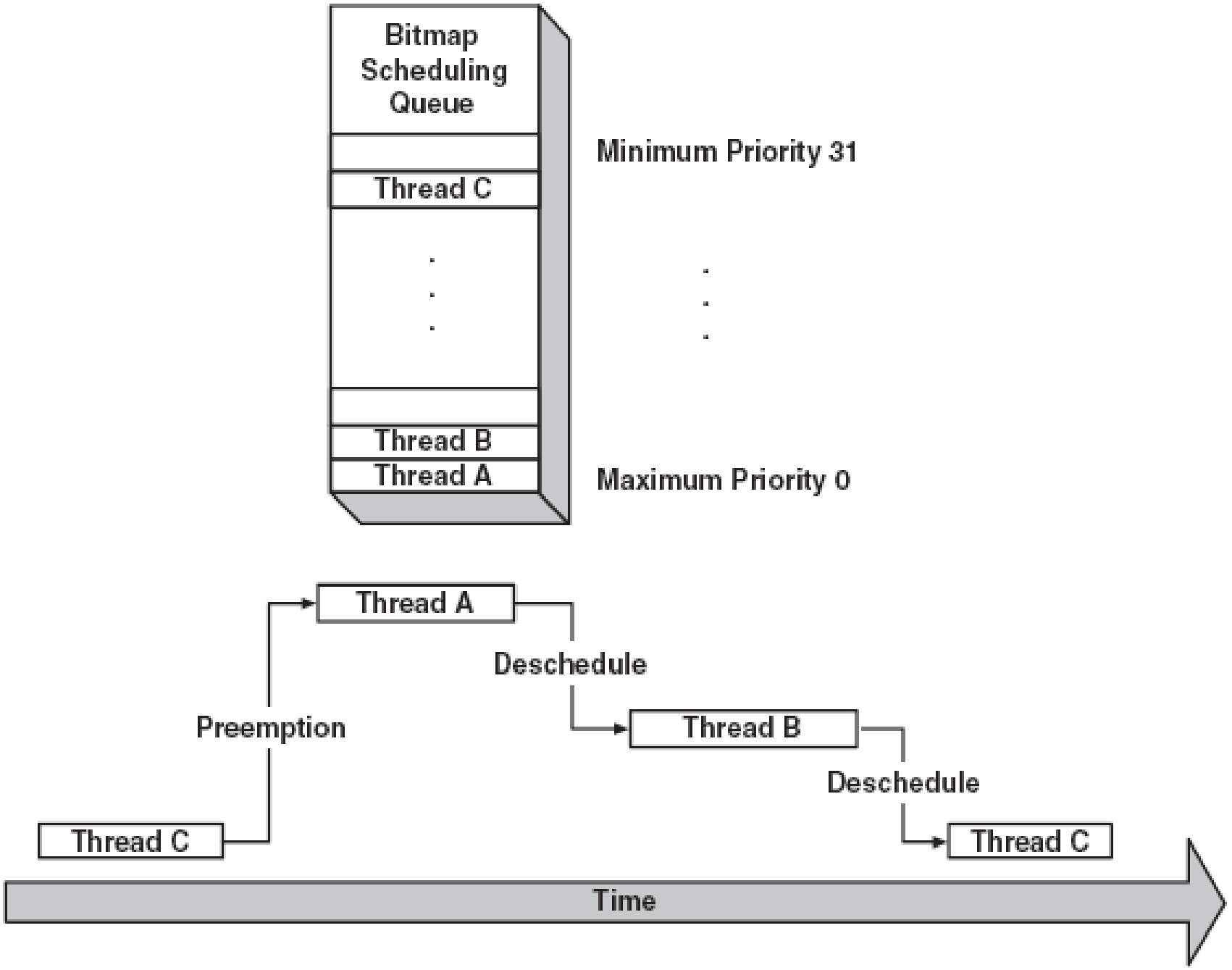
- It’s the core of the kernel.
- eCos provides two schedulers
  - Multilevel Queue Scheduler
  - Bitmap Scheduler

Multilevel Queue Scheduler

- Allows multiple threads at same priority level.
- Allows pre-emption between different priority levels.
- Timeslicing within a priority level allowed.

Bitmap Scheduler

- Only single thread at each priority level.
- Pre-emption between different priority levels allowed.
- Makes the scheduling algorithm simple and hence efficient.


Bitmap
Scheduling
Queue
Minimum Priority 31
Thread C
Thread B
Thread A
Maximum Priority 0
Thread A
Deschedule
Preemption
Thread B
Deschedule
Thread C
Thread C
Time

Threads

- eCos kernel provides API functions for controlling threads within a function.
- In addition to kernel threads eCos also allows POSIX threads.

Thread handling fuctions

- Various thread controlling functions exist to
  - create and exit threads
  - kill or delete threads
  - yield a thread.
  - delay, suspend and resume threads.
  - and more thread specific functions

Synchronization Mechanisms

- The synchronization mechanisms provided by eCos are :
  - mutexes
  - semaphores
  - condition variables
  - flags
  - message Boxes
  - spinlocks (For SMP systems)

Mutexes

- Mutexes allow multiple threads to share resources serially.
- Mutexes provide protection against ***Priority Inversion Problem***.
- eCos provides ***Priority Ceiling Protocol*** and ***Priority Inheritance protocol*** as solutions to above problem.

The Protocols

- Priority Ceiling Protocol
  - priority of the owner of mutex is raised to some predefined value.
  - not elegant.

- Priority Inheritance
  - priority of owner of thread is raised to highest level of all threads waiting for the mutex.
  - Synchronization calls are costlier.

Mutexes API

- Kernel Mutex controlling API
  - cyg_mutex_init()
  - cyg_mutex_destroy()
  - cyg_mutex_lock()
  - cyg_mutex_trylock()
  - cyg_mutex_unlock()
  - cyg_mutex_release()
  - cyg_mutex_set_ceiling()
  - cyg_mutex_set_protocol()

Semaphores

- eCos kernel provides API functions for creating and manipulating semaphores.
- Kernel API is for counting semaphores and not binary semaphores.

Semaphores API

- Kernel Semaphore controlling API
  - cyg_semaphore_init()
  - cyg_sempahore_destroy()
  - cyg_semaphore_wait()
  - cyg_semaphore_trywait()
  - cyg_semaphore_timed_wait()
  - cyg_semaphore_post()
  - cyg_semaphore_peek()

Condition Variables

- Condition variables are used with mutexes to allow multiple threads to access shared data.
- eCos kernel provides API to control condition variables.

Condition Variables API

- Kernel API to control condition variables
  - cyg_cond_init()
  - cyg_cond_destroy()
  - cyg_cond_wait()
  - cyg_cond_timed_wait()
  - cyg_cond_signal()
  - cyg_cond_broadcast